Assignment 1: Parallel Numerical Integration

CS5741 Concurrency and Parallelism in Software Development, AY26-27 Semester 1

Chirag Aggarwal · ID 26253925

chiragaggarwal5k@gmail.com

Lecturer: Faeq Alrimawi

1. Foster's Methodology

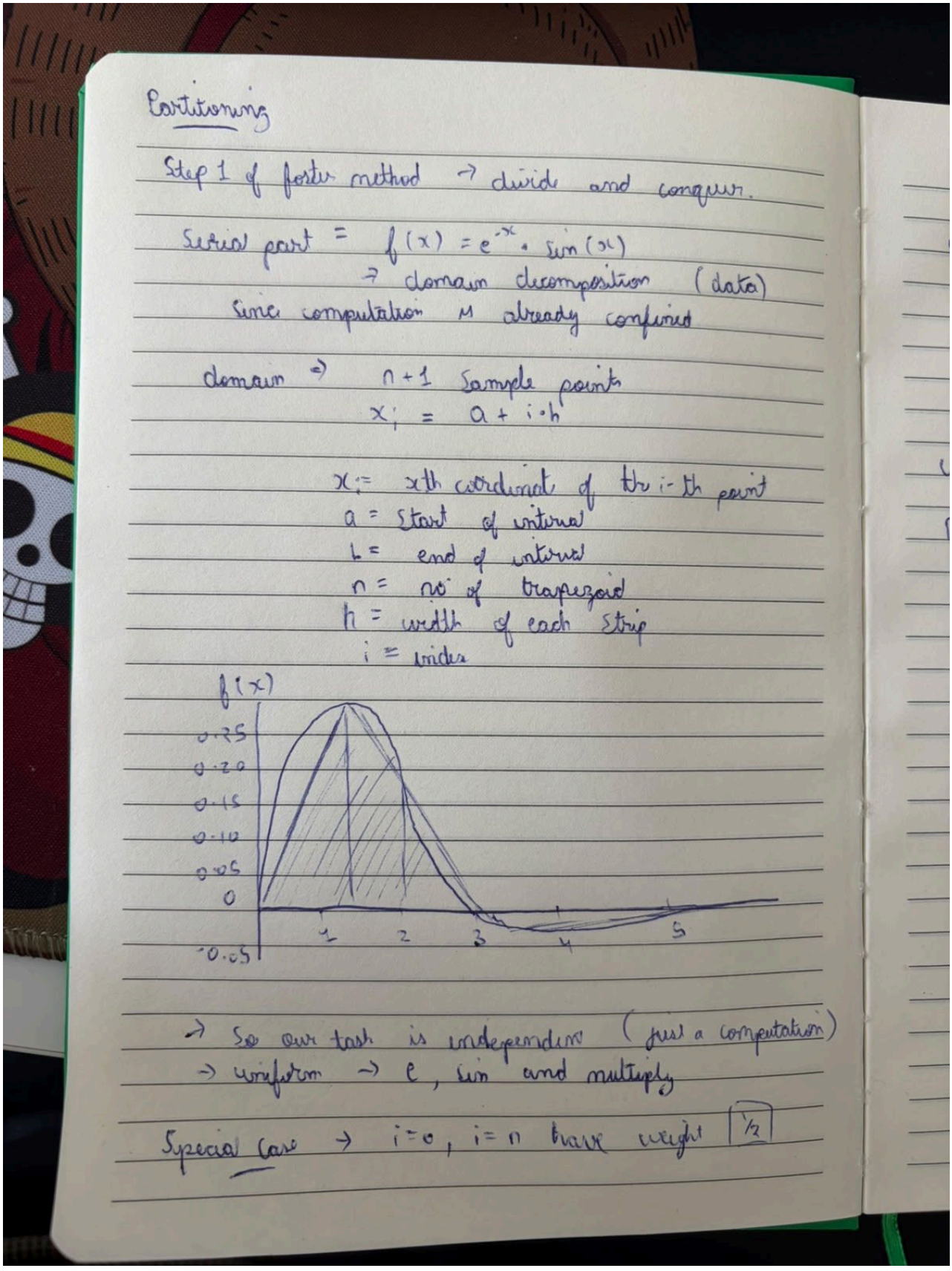


Figure 1: Partitioning: domain decomposition, variables, sketch of the tasks

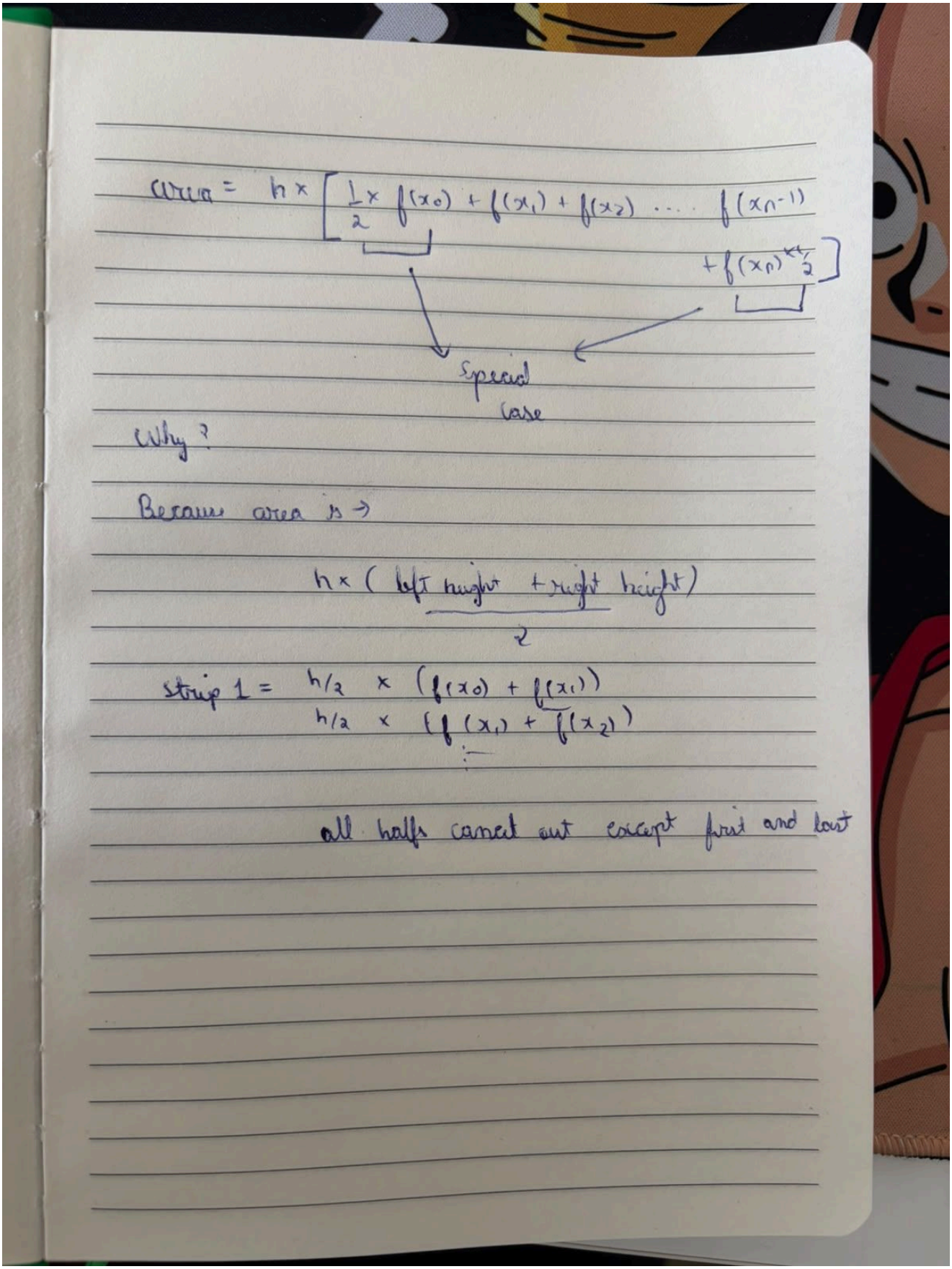


Figure 2: Partitioning: why the endpoints carry weight ½

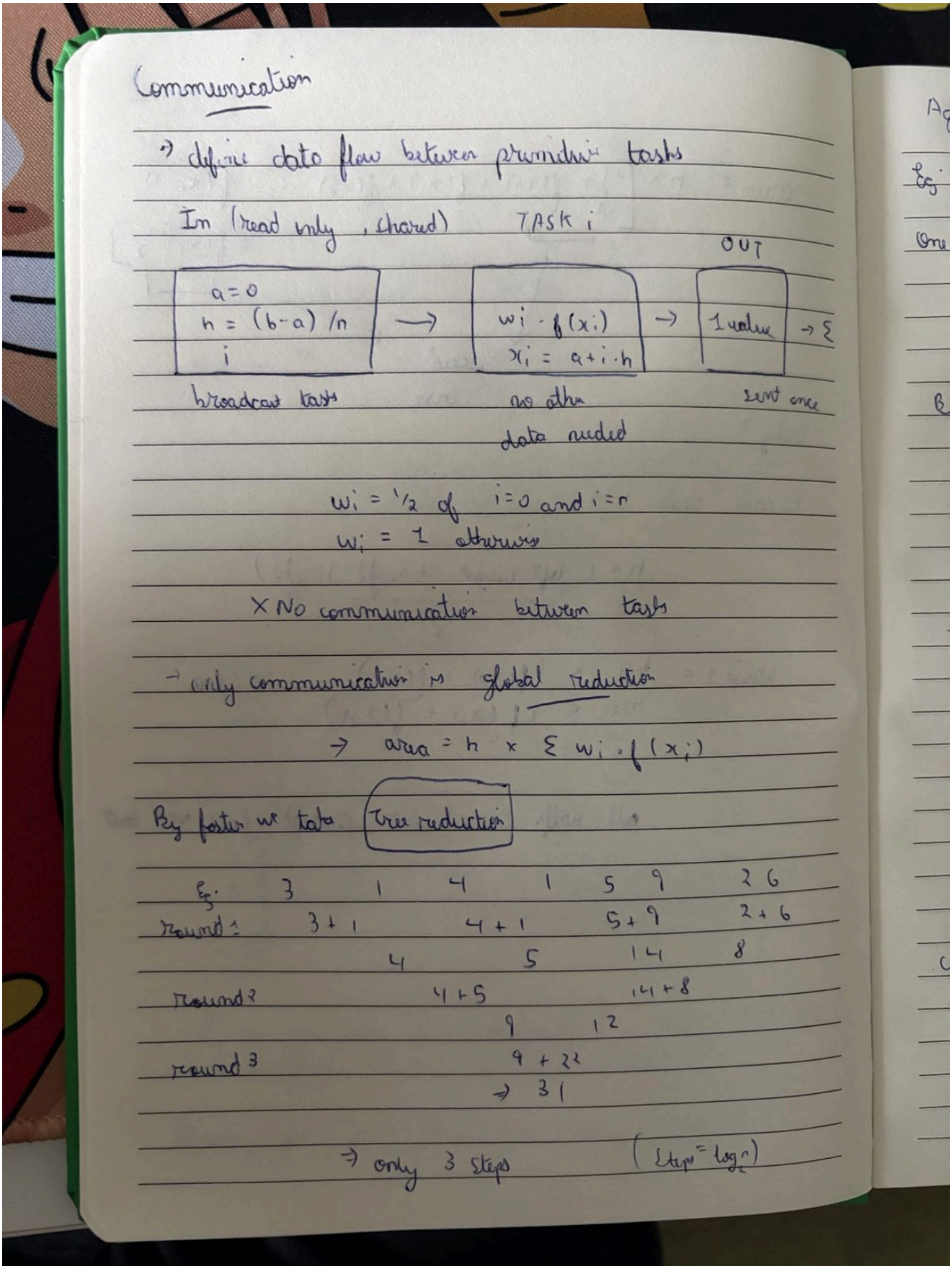


Figure 3: Communication: data flow of one task and tree reduction

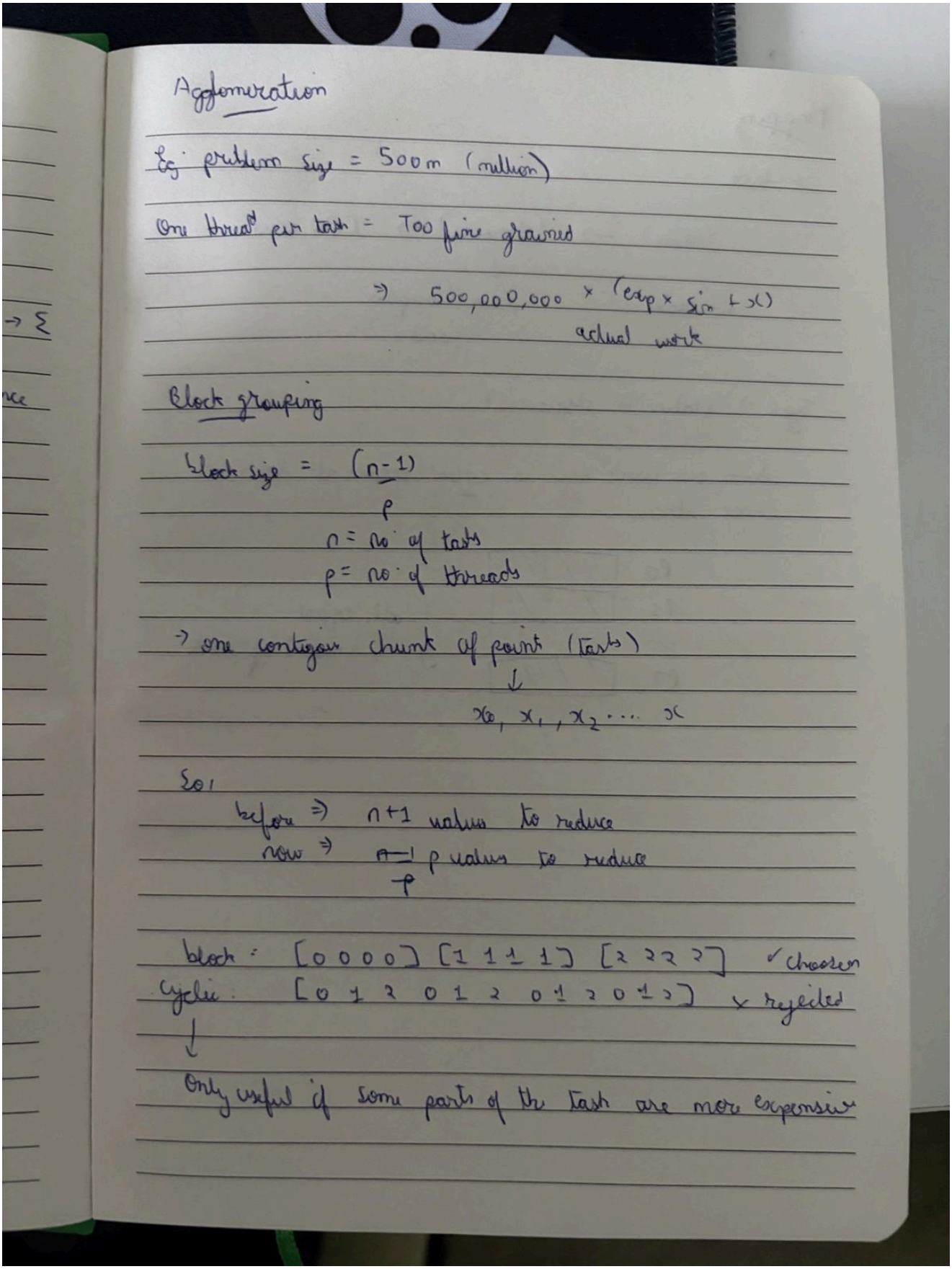


Figure 4: Agglomeration: block grouping, values to reduce, block vs cyclic

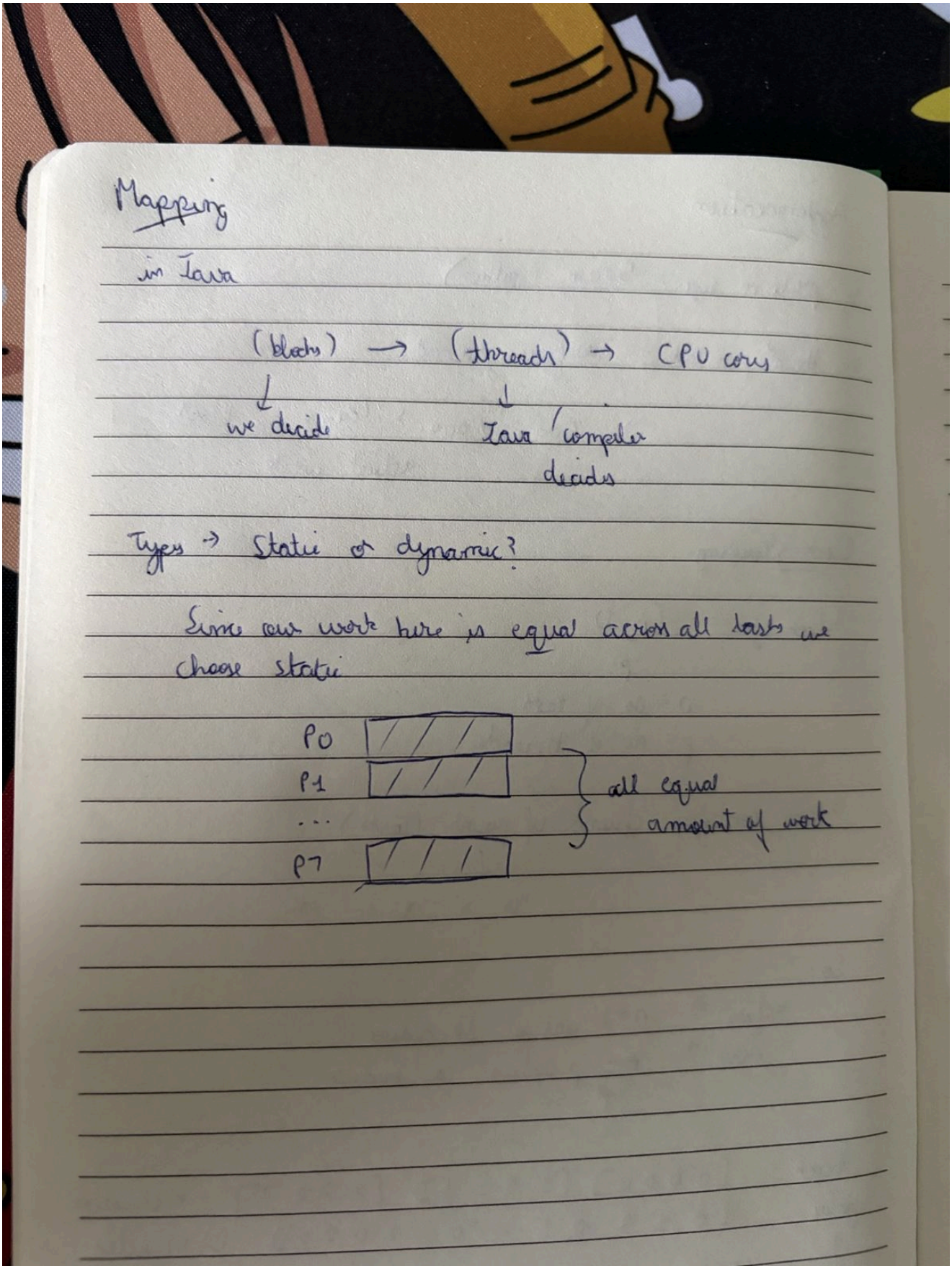


Figure 5: Mapping: blocks to threads to cores, static mapping

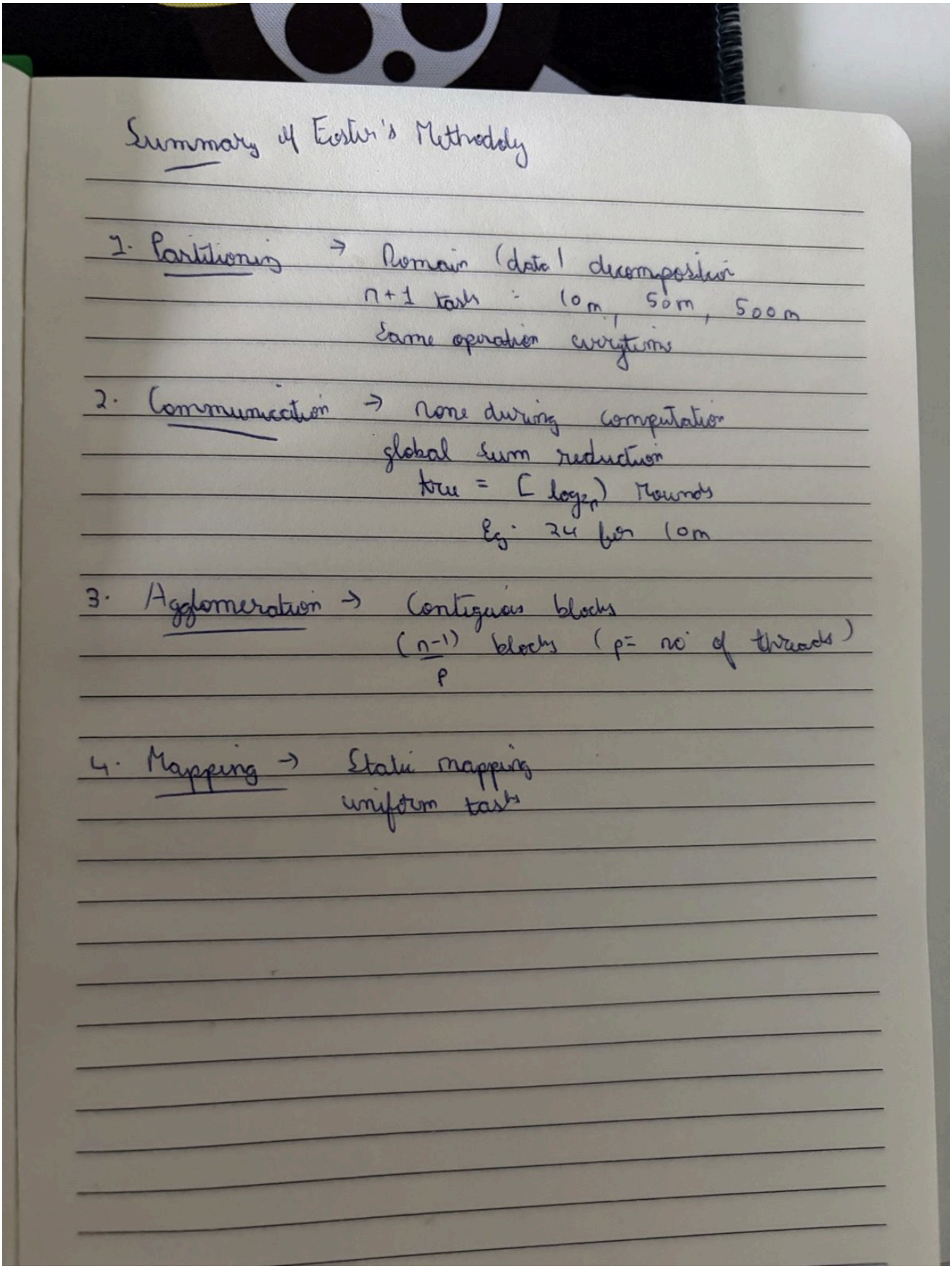


Figure 6: Summary of Foster's methodology

2. Parallel Implementation

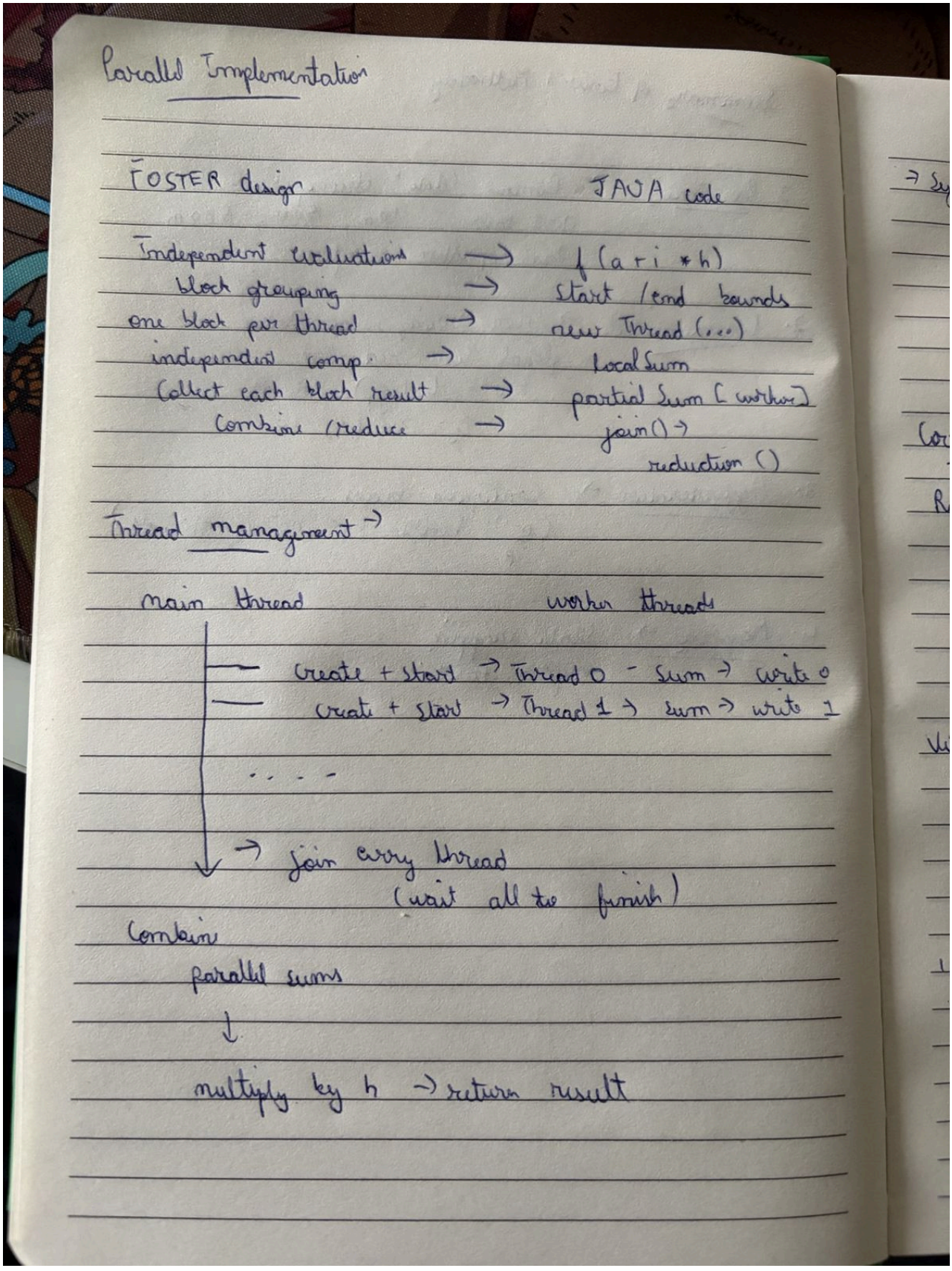


Figure 7: Parallel implementation: Foster design to Java code, thread management

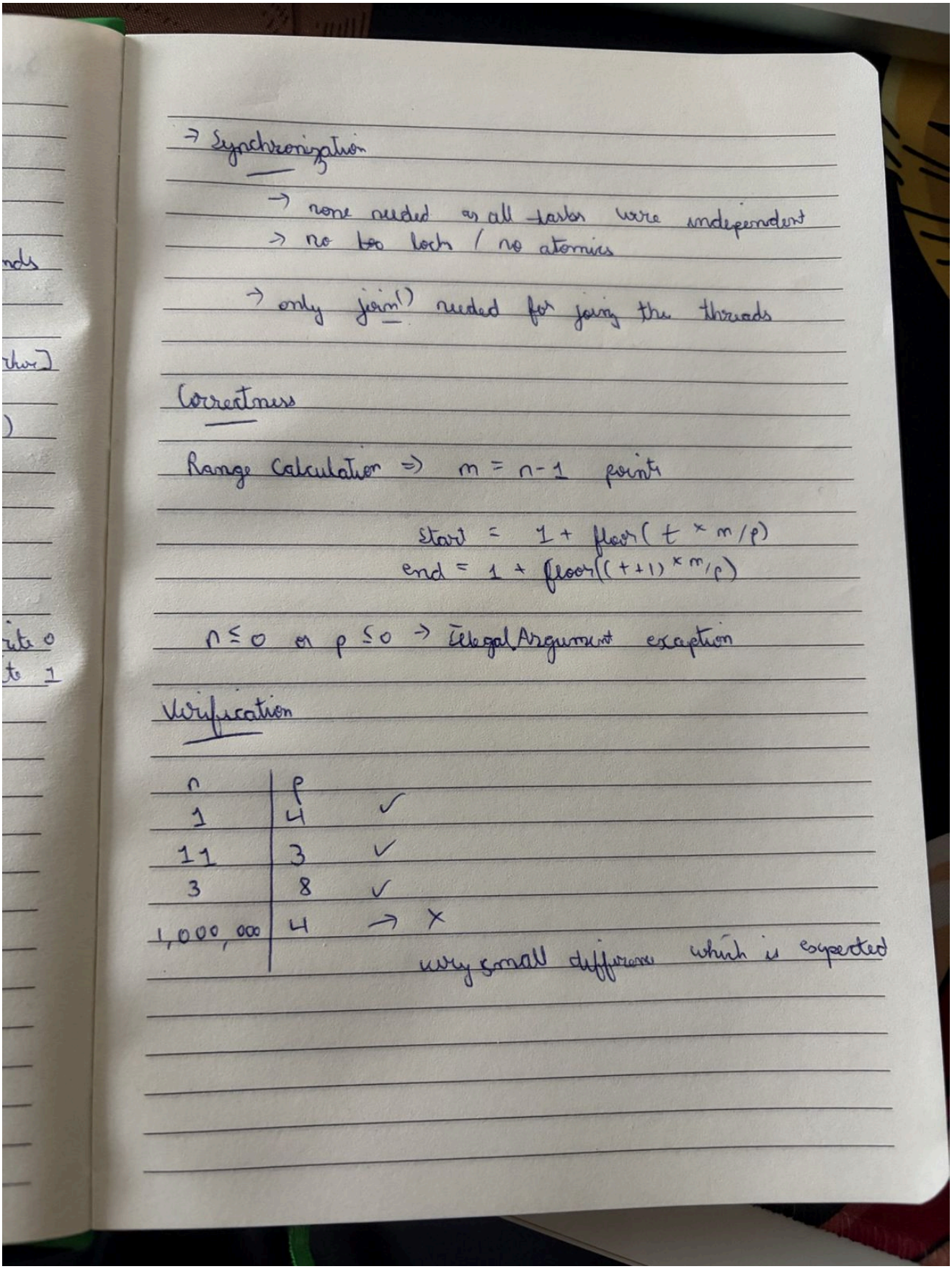


Figure 8: Parallel implementation: synchronisation, correctness, verification

3. Machine Specifications

CPU	Apple M2 Pro
Cores	12 (8 performance + 4 efficiency); each core runs one thread at a time
L2 cache	16 MB (performance cluster), 4 MB (efficiency cluster)
Memory	32 GB
OS	macOS 26.6.2
Java	OpenJDK 27

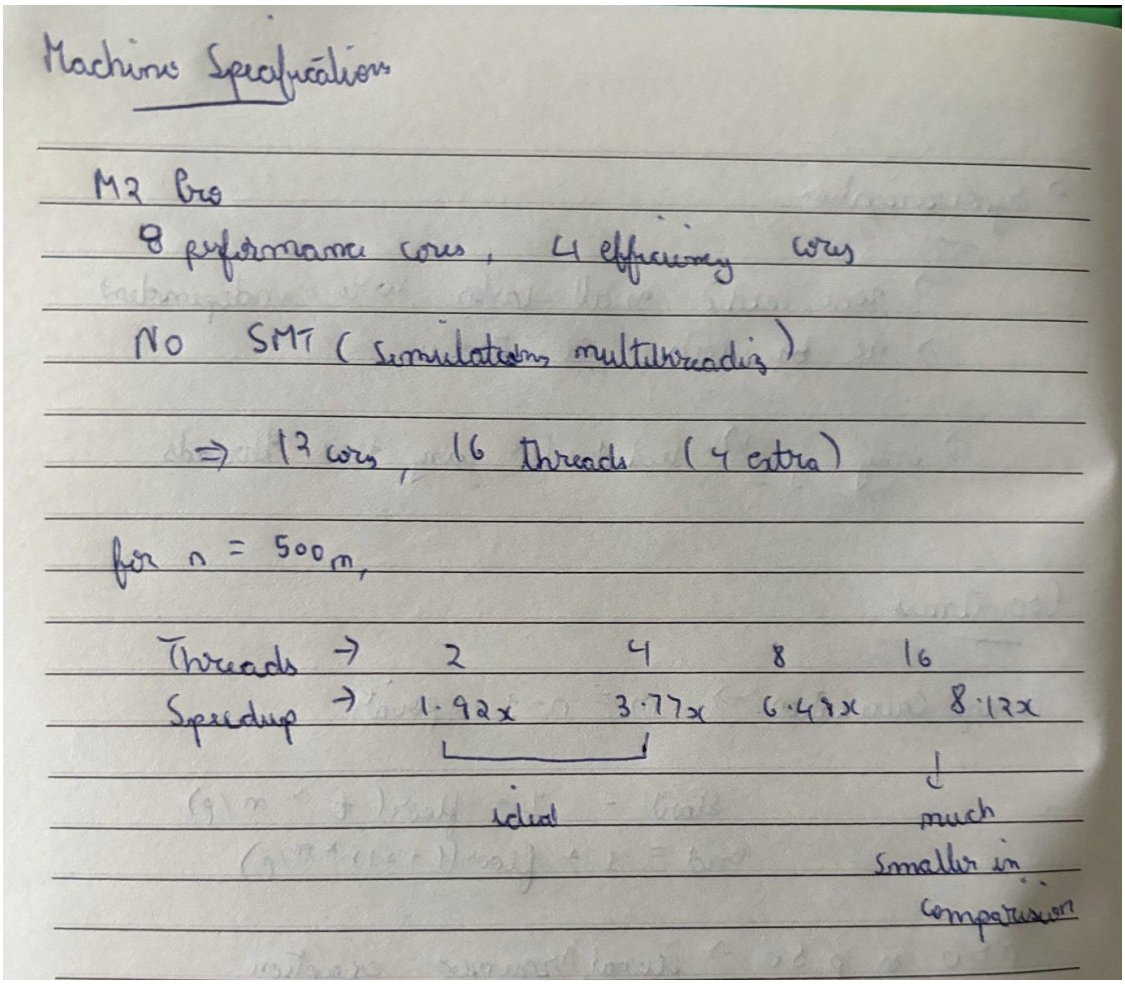


Figure 9: How the machine specifications affect speedup

4. Performance Analysis

4.1 Method

Each configuration was run 7 times after 3 warm-up rounds, and the median is reported. Runs were made on AC power.

4.2 Execution time (median, ms)

n	serial	p = 2	p = 4	p = 8	p = 12	p = 16
10M	106.4	56.4	28.4	20.2	17.5	17.6
100M	1,067.6	561.5	286.2	181.2	144.4	144.9
500M	5,362.3	2,804.1	1,458.7	865.2	691.1	684.8

4.3 Actual speedup, $S = T_{serial} / T_{parallel}$

n	p = 2	p = 4	p = 8	p = 12	p = 16
10M	1.89	3.74	5.26	6.07	6.03
100M	1.90	3.73	5.89	7.39	7.37
500M	1.91	3.68	6.20	7.76	7.83

4.4 Efficiency, S / p

n	p = 2	p = 4	p = 8	p = 12	p = 16
10M	94%	94%	66%	51%	38%
100M	95%	93%	74%	62%	46%
500M	96%	92%	77%	65%	49%

4.5 Fixed thread overhead (n = 1, every range empty; median of 200 runs, ms)

p = 2	p = 4	p = 8	p = 12	p = 16
0.211	0.311	0.567	0.868	1.151

4.6 Amdahl's Law

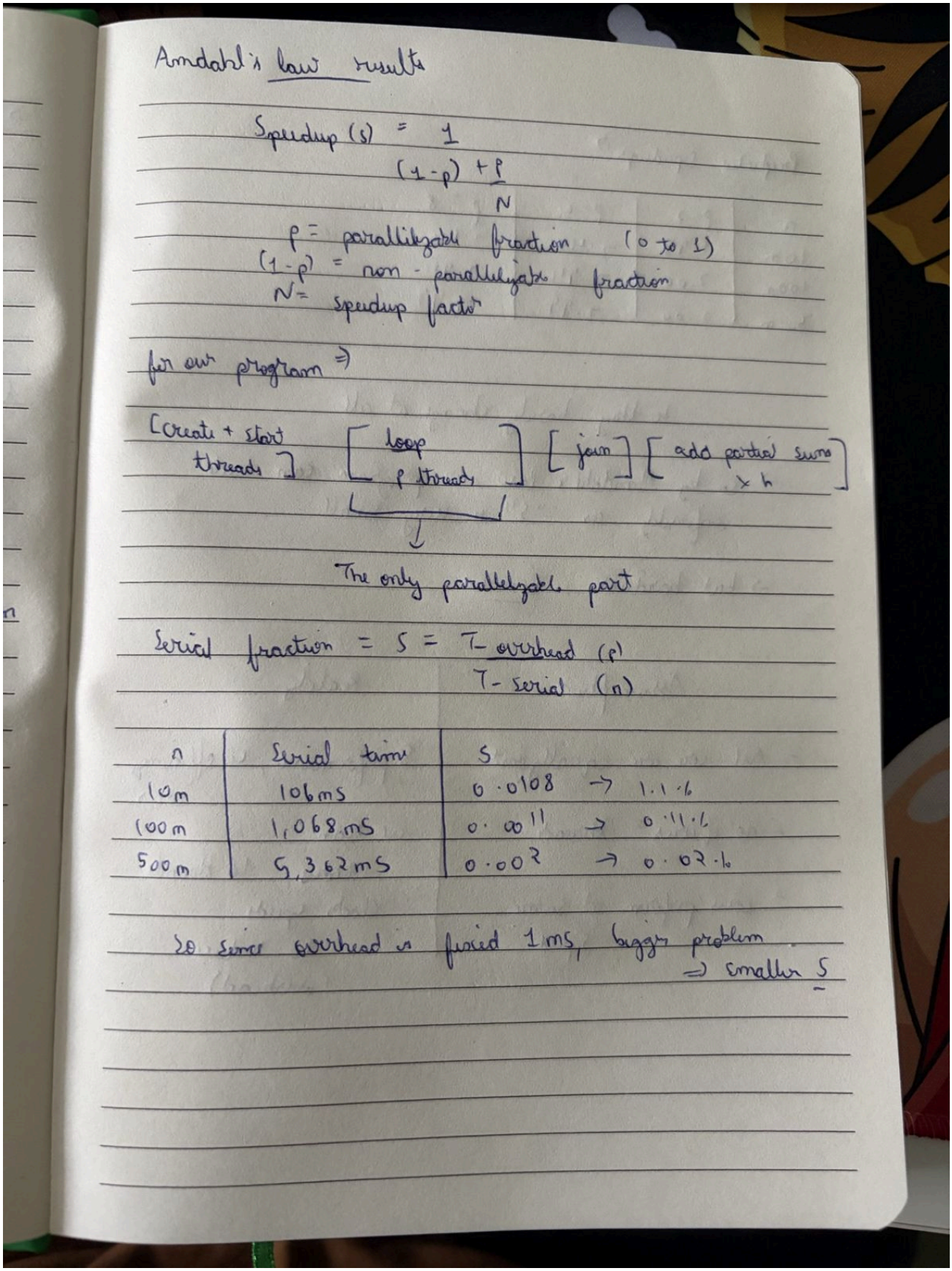


Figure 10: Amdahl's Law: the formula, the serial part of the program, and the serial fraction s

Predicted Speedups →

n	p=2	p=4	p=8
10m	1.98	7.44	7.44
100m	2.00	7.94	7.94
500m	2.00	7.99	7.99

so they barely change at all

→ so by Amdahl's law our program can scale infinitely as "S" is very small

→ but hardware is limited

Assumption	Reality
- All core are equally fast	8 performance, 4 efficiency cores
- no of cores = threads	we ran 12 cores, 16 threads
- cores perform at same speed	Clocks speeds do reduce (overhead)

Figure 11: Amdahl's Law: predicted speedups, and the assumptions that do not hold on this machine

4.7 Graphs

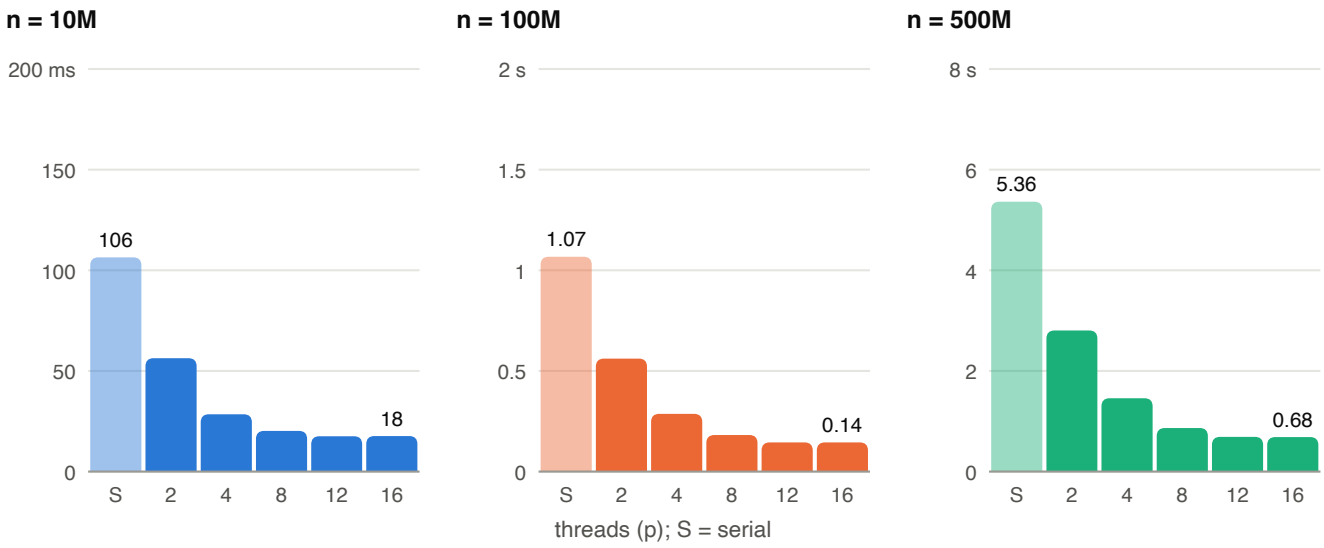


Figure 12: Median execution time for each problem size (S = serial). Each panel has its own scale.

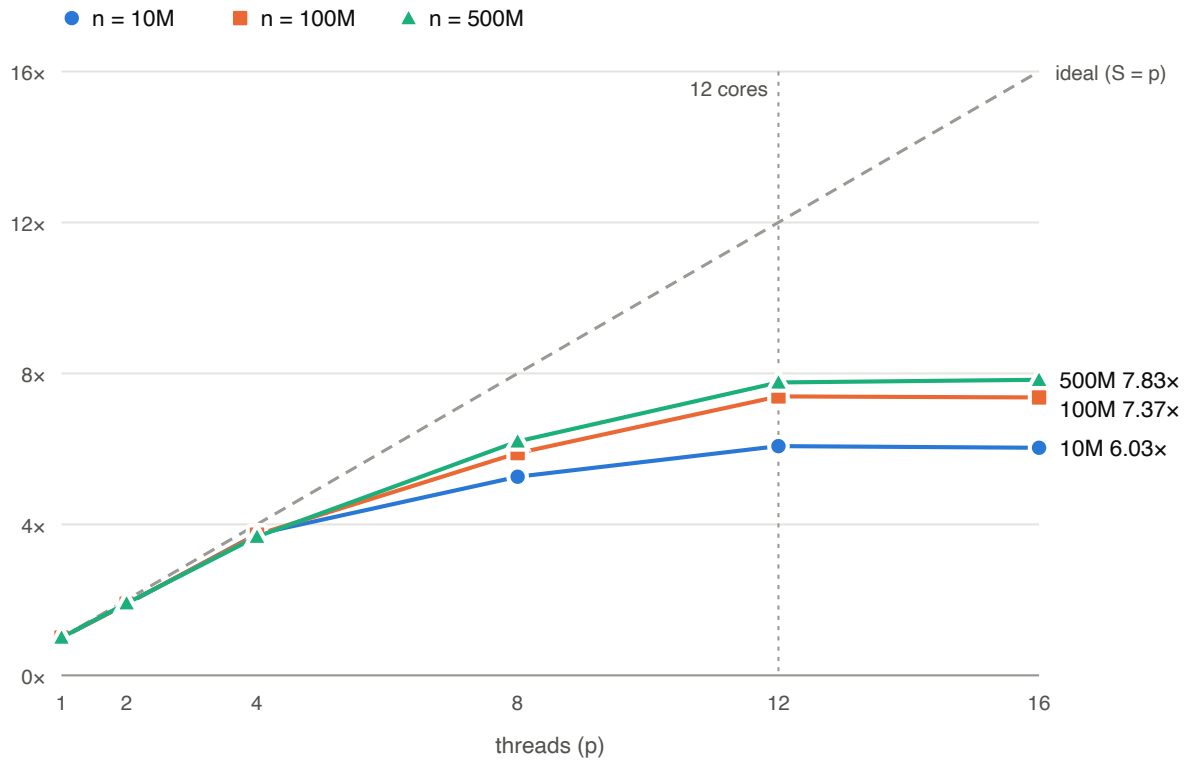


Figure 13: Actual speedup against thread count, with ideal linear speedup and the 12-core limit.

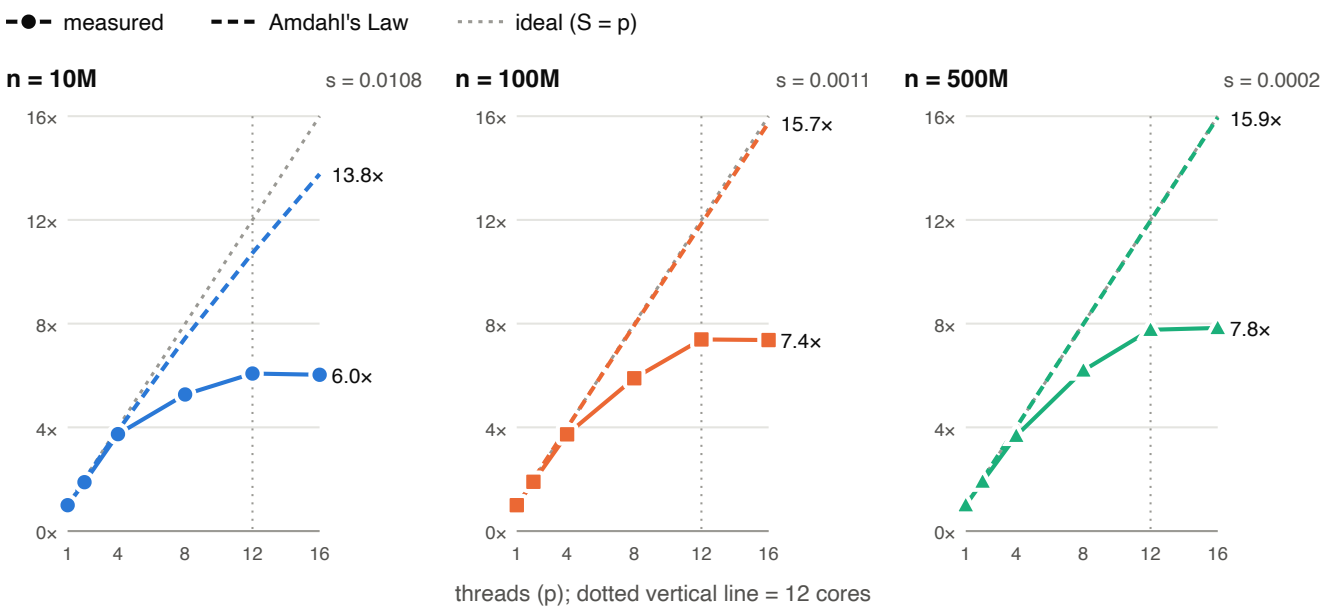


Figure 14: Measured speedup against Amdahl's Law for each problem size, using $s = T_{overhead}(p = 16) / T_{serial}(n)$. At 100M and 500M the Amdahl line lies almost on top of the ideal line.

4.8 Discussion

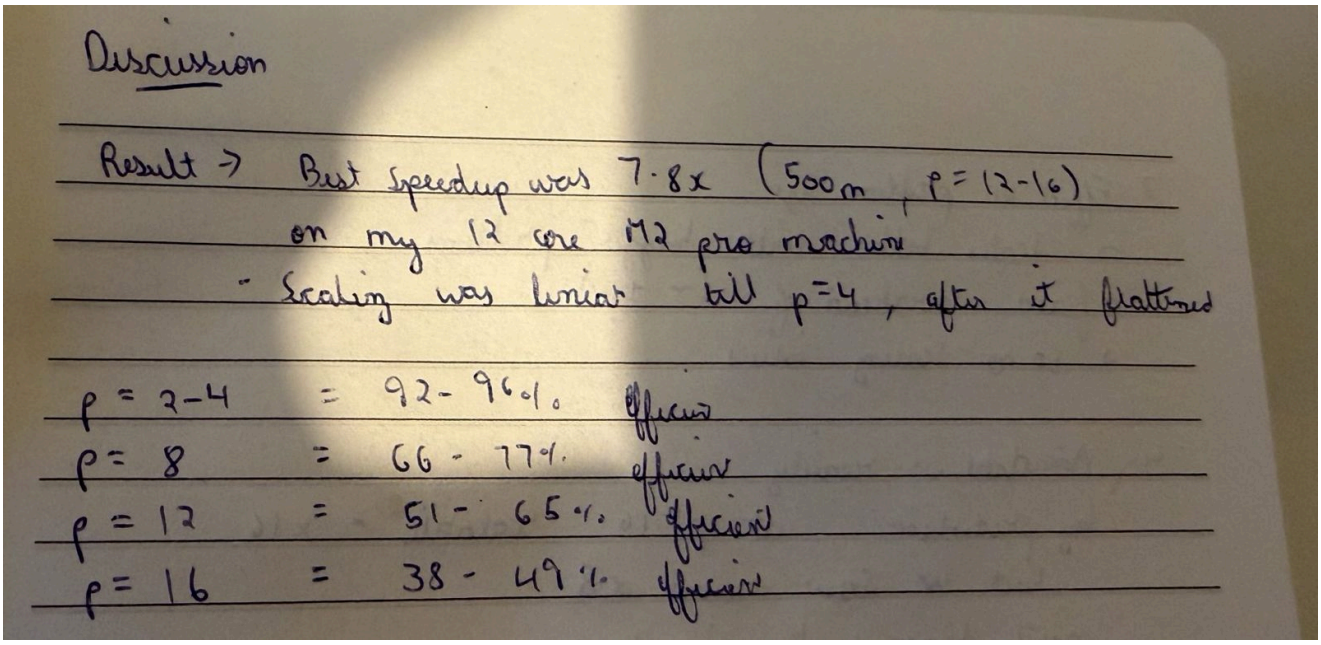


Figure 15: Discussion: overall result and efficiency at each thread count

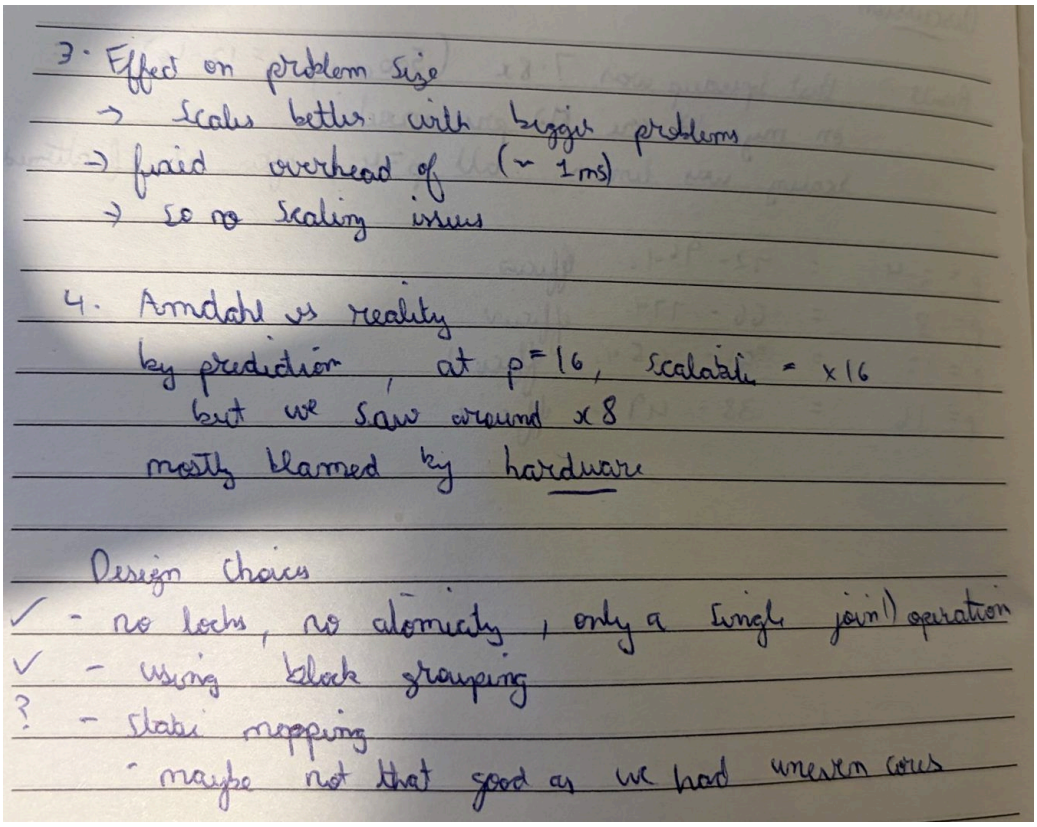


Figure 16: Discussion: problem size, Amdahl vs reality, and evaluation of the design choices

5. Conclusion

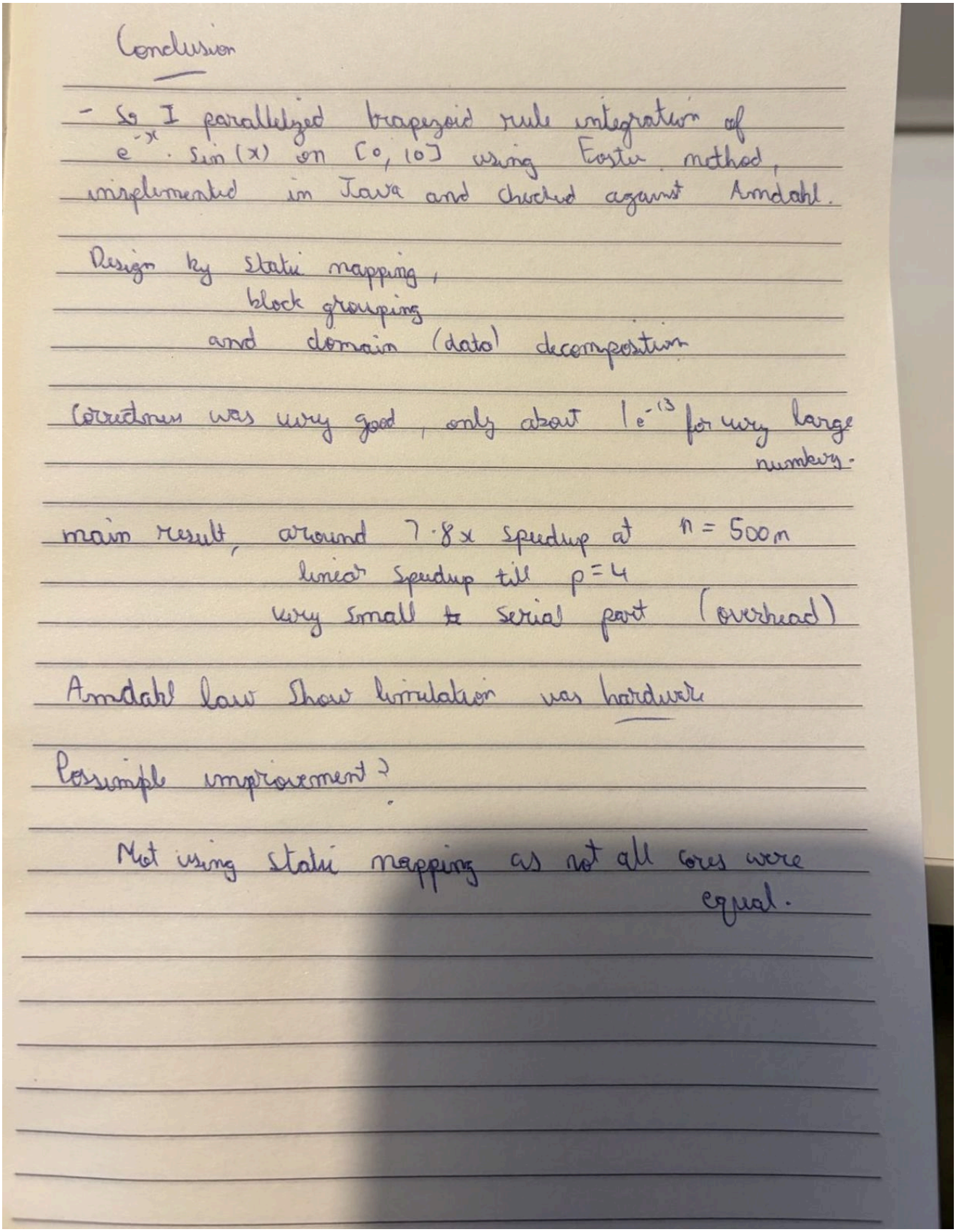


Figure 17: Conclusion

References

1. Alrimawi, F. (2026a) *CS5741 Assignment 1: Parallel Numerical Integration* [assignment brief and supplied `SerialIntegration.java`]. University of Limerick, Brightspace, 8 October.
   Task specification, marking criteria, the trapezoidal rule and the serial baseline code.

2. Alrimawi, F. (2026b) *Lecture 2: Identifying Parallelisable Problems* [lecture slides], CS5741. University of Limerick.
   Decomposability, independence and granularity; data vs task parallelism; Amdahl's Law (slides 42–47); strong vs weak scaling (slide 48).

3. Alrimawi, F. (2026c) *Lecture 3: Parallel Algorithms* [lecture slides], CS5741. University of Limerick.
   ForkJoin and estimating the serial fraction (slides 9–10); task dependencies (RAW/WAR/WAW); static vs dynamic load balancing; Foster's methodology and its four checklists (slides 24–39); parallel patterns (slide 40).

4. Alrimawi, F. (2026d) *Lab 1: Sequential vs Concurrent vs Parallel* [lab handout and code], CS5741. University of Limerick.
   Shown in practice: a shared, synchronised accumulator serialises the threads.

5. Amdahl, G.M. (1967) 'Validity of the single processor approach to achieving large scale computing capabilities', in *Proceedings of the AFIPS Spring Joint Computer Conference*, Atlantic City, 18–20 April, pp. 483–485. doi:10.1145/1465482.1465560.
   Original statement of the speedup limit used in Section 4.

6. Apple Inc. (2023) 'Apple unveils M2 Pro and M2 Max: next-generation chips for next-level workflows', *Apple Newsroom*, 17 January. Available at: https://www.apple.com/newsroom/2023/01/apple-unveils-m2-pro-and-m2-max-next-generation-chips-for-next-level-workflows/ (accessed 14 October 2026).
   Core configuration of the test machine (performance and efficiency cores).

7. Foster, I. (1995) *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Reading, MA: Addison-Wesley.
   Source of the four-stage design methodology (partitioning, communication, agglomeration, mapping) applied in Section 1.

8. Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D. and Bierman, G. (2023) *The Java Language Specification, Java SE 21 Edition*, §17.4.4–17.4.5. Oracle. Available at: https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html (accessed 14 October 2026).
   Memory-model guarantee that all actions in a thread happen-before another thread returns from `join()` on it, which is why the partial sums are visible without locks.

9. Grama, A., Gupta, A., Karypis, G. and Kumar, V. (2003) *Introduction to Parallel Computing*. 2nd edn. Harlow: Addison-Wesley.
   Decomposition techniques (recursive, data, exploratory, speculative), used when considering alternative partitions.

10. Oracle (2023) *Class Thread*, Java SE 21 API documentation. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html (accessed 14 October 2026).
    `Thread` creation, `start()` and `join()` as used in the implementation.

11. Anthropic (2026) *Claude* [AI assistant]. Available at: https://claude.ai (accessed 10–14 October 2026).
    Used within the limits of section 4 of the brief; see the AI Usage Statement for exactly how.

AI Usage Statement

I used Claude (Anthropic, 2026), an AI assistant, during this assignment within the limits set in section 4 of the brief. This is how it was used.

Area	What the AI did	What I did
Understanding the problem	Explained the trapezoidal rule, the integrand $e^{-x}\cdot\sin(x)$, the endpoint weights and terms such as global reduction, block grouping and SMT. Built two interactive explainer pages (the curve with its trapezoids, and agglomeration).	Studied the explanations and the lecture slides.
Foster's methodology (Section 1)	Gave outlines for all four stages, cross-checked against the Lecture 3 checklists. Wrote example drafts for the Partitioning and Communication stages, and reviewed my Partitioning notes.	Handwrote every page in Figures 1–6 in my own words, including the diagrams.
Parallel implementation (Section 2, Java file)	Supplied a skeleton: the header comment, the supplied serial method, the timing and printing in `main`, and numbered TODO comments. Reviewed my finished code and tested it against the serial version on 64 (n, p) combinations.	Wrote the `parallel()` method: range bounds, thread creation, local sums, `join()` and the reduction. Handwrote Figures 7–8.
Machine specifications (Section 3)	Read the hardware details from the machine and explained what they mean.	Handwrote Figure 9.
Performance measurement (Section 4)	Wrote and ran the benchmark harness (warm-up, median of 7 runs, the thread-overhead measurement at n = 1). Generated the result tables and Figures 10–11 from the measured data.	Chose the experiments, including the extra p = 12 comparison.
Amdahl's Law (Section 4.6)	Explained the law, and calculated the serial fraction s, the predicted speedups and the maximum speedup from the measured data.	Handwrote Figures 10–11, including the comparison of Amdahl's assumptions with the real hardware.
Discussion and conclusion (Sections 4.8 and 5)	Gave outlines listing the points to cover and the supporting numbers from the results.	Handwrote Figures 15–17.
Report production	Downloaded the brief and lecture slides from Brightspace, assembled this PDF from my scanned pages, cropped the scans, and formatted the reference list.	Decided the structure and content of the report.

I checked all AI output for accuracy before using it, and I can explain every part of the design, the code and the results.